

SITS — Strategic Interaction Tournament Simulator

1. Problem Introduction

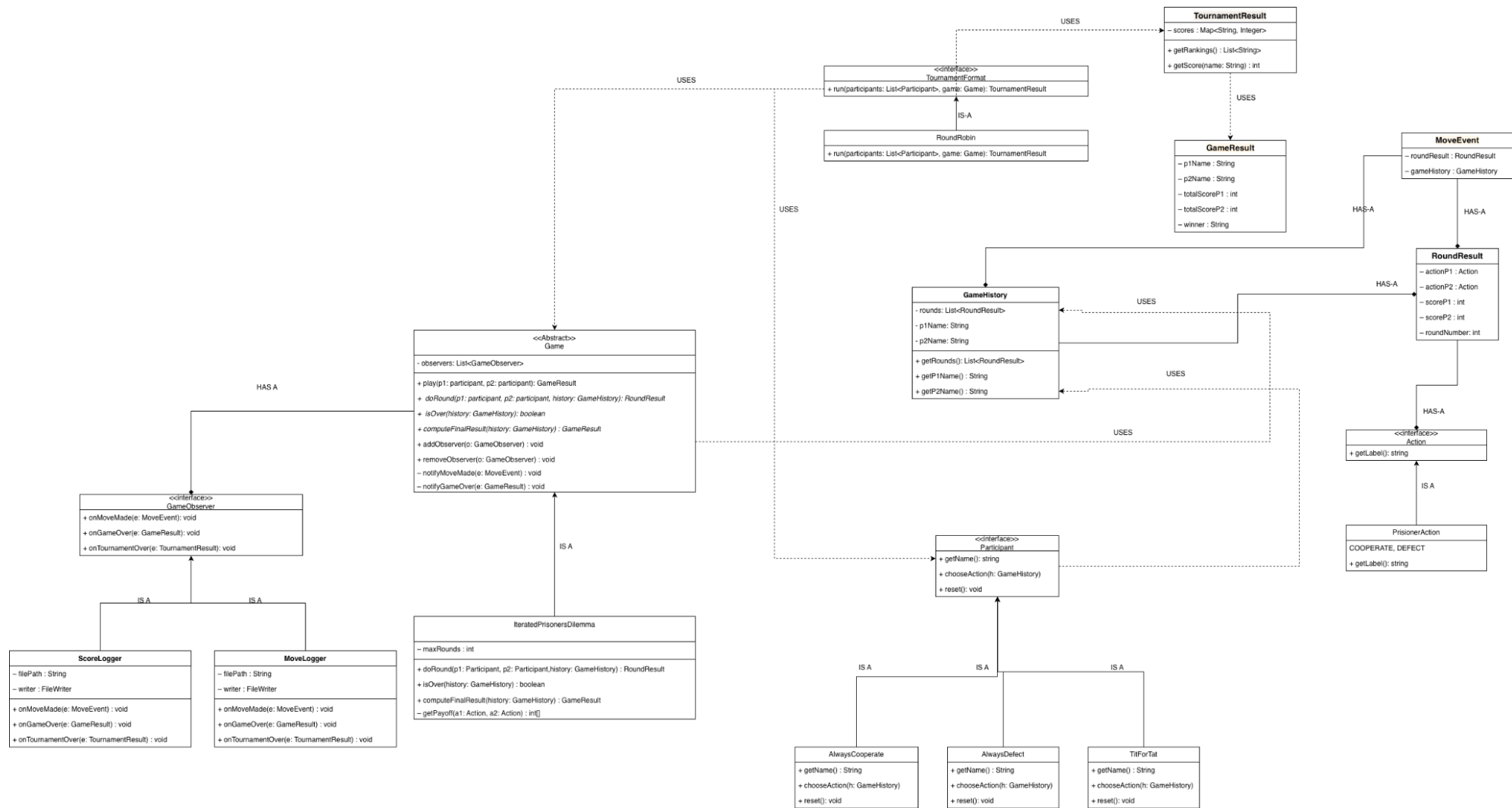
A research organization needs a flexible simulation platform to study how decision-making strategies perform in repeated competitive situations. The platform must allow researchers to pit automated agents against one another across a variety of games and tournament formats, and to observe and log the results without having to rebuild the system each time a new game, strategy, or tournament format is introduced.

The core challenge is that the system must be open to extension but closed to modification. Researchers are not software engineers — they should be able to introduce a new game like Rock Paper Scissors, or a new participant strategy, by writing a small, self-contained piece of code without touching anything that already works. The system must also support flexible logging so that results can be recorded in different formats and eventually displayed in a GUI.

This document describes the design decisions made to meet those requirements, the rationale behind tricky relationships, and the alternatives that were considered and rejected.

2. UML Class Diagram

[See next page



3. Design Patterns

3.1 Template Method — `Game`

The abstract `Game` class defines a fixed game loop in its `play()` method. This method calls three abstract methods that subclasses must implement: `doRound()`, `isOver()`, and `computeFinalResult()`. The loop itself — calling rounds, checking termination, accumulating history, notifying observers — never changes regardless of the game being played.

This pattern was chosen because all two-player turn-based games share the same structural loop.

3.2 Strategy Pattern — `Participant` and `TournamentFormat`

Every participant implements the `Participant` interface, which exposes a single decision method: `chooseAction(GameHistory history)`. Each concrete participant encapsulates a completely different decision strategy.

The same pattern applies to tournament formats. `TournamentFormat` is an interface with a single `run()` method. `RoundRobin` is the first implementation. Future formats like Double Elimination can be added without touching the tournament runner or any game code.

3.3 Observer Pattern — `GameObserver`

The `Game` abstract class maintains a list of `GameObserver` instances. Whenever a move is made or a game ends, it fires events to all registered observers. `MoveLogger` and `ScoreLogger` are two concrete observers that write to separate files. A future GUI observer could register itself the same way with zero changes to game or tournament code.

This pattern was specifically required by the assignment and is the right tool here because logging is a cross-cutting concern that should have no footprint inside the game logic itself.

4. Tricky Relationships and Rationale

4.1 `Action` as an Interface

`Action` is defined as a plain interface with a single method:

```
public interface Action {
```

```
String getLabel();  
  
}
```

Game implementers provide their own enums that implement this interface:

```
public enum PrisonerAction implements Action {  
  
    COOPERATE, DEFECT;  
  
    @Override  
  
    public String getLabel() { return name(); }  
  
}
```

Why not a single shared enum?

A shared `Action` enum with `COOPERATE`, `DEFECT`, `ROCK`, `PAPER`, `SCISSORS` would require modifying the core framework every time a new game is added.

4.2 Game HAS-A `List<GameObserver>`

The game does not own its observers — observers are registered from outside and can be detached. This is aggregation. If the game were destroyed, the loggers would still exist and could be reattached to a new game. This matters for long-running research sessions where multiple games run sequentially and share the same log files.

4.3 `RoundResult` Stores `Action` (the Interface), not `PrisonerAction`

`RoundResult` holds `actionP1 : Action` and `actionP2 : Action`, not concrete enum types. This means `GameHistory`, `MoveEvent`, and all observers handle actions generically. A `MoveLogger` can call `action.getLabel()` to record what happened without importing or knowing about `PrisonerAction` or any other game-specific type. The entire logging infrastructure is decoupled from every game.

4.4 `Participant.chooseAction()` Receives `GameHistory`, not Just the Last Move

Passing the full `GameHistory` instead of just the opponent's last action gives participants everything they could ever need. `AlwaysCooperate` ignores it entirely. `TitForTat` reads only the last round. A future strategy could analyze the full history for patterns. This design does not

force participants to hold an internal state — all the information they need is in the history object passed to them each round. (I think this could be a dependency injection)

5. Tricky Code

5.1 The `getPayoff()` Method in `IteratedPrisonersDilemma`

```
private int[] getPayoff(Action a1, Action a2) {  
  
    if (a1 == PrisonerAction.COOPERATE && a2 == PrisonerAction.COOPERATE) return new  
int[]{3, 3};  
  
    if (a1 == PrisonerAction.COOPERATE && a2 == PrisonerAction.DEFECT) return new  
int[]{0, 5};  
  
    if (a1 == PrisonerAction.DEFECT && a2 == PrisonerAction.COOPERATE) return new  
int[]{5, 0};  
  
    if (a1 == PrisonerAction.DEFECT && a2 == PrisonerAction.DEFECT) return new int[]{1,  
1};  
  
    throw new IllegalArgumentException(  
  
        "Unrecognized actions: " + a1.getLabel() + ", " + a2.getLabel()  
  
    );  
}
```

This is the only place in the entire system that references `PrisonerAction` by name. The method takes `Action` parameters (the interface) but casts down to concrete enum values for comparison. The `IllegalArgumentException` at the end is intentional — if a wrong action type is passed in (e.g., an `RPSAction` passed to a Prisoner's Dilemma game), the system fails loudly with a clear error message rather than silently producing wrong scores.

5.2 `TitForTat.chooseAction()`

`@Override`

```
public Action chooseAction(GameHistory history) {  
  
    if (history.getRounds().isEmpty()) {
```

```

        return PrisonerAction.COOPERATE; // cooperate on first round
    }

    return history.getLastRound().getActionP2(); // mirror opponent's last move
}

```

The edge case here is the first round — there is no previous move to mirror, so `TitForTat` defaults to cooperation.

5.3 Observer Notification in `Game.play()`

```

public GameResult play(Participant p1, Participant p2) {

    GameHistory history = new GameHistory(p1.getName(), p2.getName());

    while (!isOver(history)) {

        RoundResult round = doRound(p1, p2, history);

        history.getRounds().add(round);

        notifyMoveMade(new MoveEvent(round, history)); // notify all observers
    }

    GameResult result = computeFinalResult(history);

    notifyGameOver(result); // notify all observers

    return result;
}

```

The notification happens after each round result is added to history, meaning observers always receive a `MoveEvent` with a complete and up-to-date `GameHistory`. This is important for any observer that wants to analyze trends mid-game rather than just recording the last move.

6. Workflow

The following describes how the system flows from start to finish during a typical research run:

1. **Setup** — A researcher instantiates a `Game` (e.g., `new IteratedPrisonersDilemma(200)`), a list of `Participant` implementations, and a `TournamentFormat` (e.g., `new RoundRobin()`). Loggers are created and registered on the game via `addObserver()`.
2. **Tournament starts** — `RoundRobin.run(participants, game)` is called. It generates every unique pair of participants.
3. **Match begins** — For each pair, `game.play(p1, p2)` is called. Both participants are reset via `reset()` first to clear any internal state.
4. **Round loop** — `play()` calls `isOver()` each iteration. While false, it calls `doRound()` which asks each participant for their action via `chooseAction(history)`, computes the payoff via `getPayoff()`, and returns a `RoundResult`.
5. **Observer notification** — After each round, `notifyMoveMade()` fires a `MoveEvent` to all registered observers. `MoveLogger` writes the round to its file. `ScoreLogger` ignores it.
6. **Game ends** — When `isOver()` returns true, `computeFinalResult()` tallies scores and produces a `GameResult`. `notifyGameOver()` fires. `ScoreLogger` writes the result to its file.
7. **Tournament ends** — After all pairs have played, `RoundRobin` assembles a `TournamentResult` from the accumulated `GameResult` objects and returns it. `notifyTournamentOver()` fires on all observers.

7. Alternatives Considered

7.1 Putting Payoff Logic Inside `Action`

One approach considered was giving `Action` a method like `getPayoff(Action opponent) : int[]`, making each action responsible for computing its own score against another action.

This was rejected for three reasons. First, it violates the Single Responsibility Principle — an action is a choice, not a scoring engine. Second, it couples actions to each other: `COOPERATE`

would need to know about `DEFECT` to return the correct score. Third, it removes the game's authority over its own rules — a researcher who wants to adjust the payoff matrix would have to edit enum values rather than a single method in the game class.

7.2 A Single Shared `Action` Enum

A single enum containing `COOPERATE`, `DEFECT`, `ROCK`, `PAPER`, and `SCISSORS` would work in the short term but requires modifying core framework code every time a new game is added. This makes the framework fragile and violates the open/closed principle. The interface approach pushes action definition responsibility entirely to the game implementer.

7.3 Participant Holding Internal State Instead of Reading `GameHistory`

An alternative design would have each participant manage its own memory of past moves internally, with `chooseAction()` taking no arguments. This was rejected because it makes `reset()` critical and error-prone — forgetting to call it between matches would corrupt the next game. Passing `GameHistory` externally makes participants stateless by default, with `reset()` as a safety net only for participants that explicitly choose to hold state.

7.4 Using `String` Instead of `Action` Interface

Using `String` as the action type would be the simplest possible approach — no interface, no enum, just `"COOPERATE"` or `"DEFECT"`. This was rejected because it provides no type safety, no compiler enforcement of valid actions, and no natural extension point. Any typo in a string would produce a silent runtime failure rather than a compile-time error.